

TOPIC 01: NODE.JS WITH MYSQL & ACCESS METHODS

Node.js + MySQL 3-Tier Architecture (Flowchart):

Communication pipeline between Client, Server, and MySQL Database:



4 Primary Methods to Access MySQL Database:

- **1. MySQL Workbench (GUI):** Desktop visual tool providing a graphical UI to design schemas, manage tables, and run queries.
- **2. MySQL Command Line Interface (CLI):** Fast terminal-based execution by authenticating via `mysql -u root -p`.
- **3. MySQL2 Package In Node.js:** Programmatically executing SQL queries directly from Express/Node.js backend applications.
- **4. External SQL Script File (schema.sql):** Executing predefined SQL scripts via the terminal using `source filename.sql`.



Core Concept: Node.js cannot communicate directly with MySQL out of the box; it relies on a **Database Driver** (like `mysql2`) to translate JavaScript calls into raw MySQL binary protocol!

TOPIC 02: FAKER PACKAGE (MOCK DATA GENERATION)

What is the Faker Package & Why is it Needed?

During testing and development, manually creating 100 or 1,000 realistic user records is impractical. The **Faker package** generates realistic dummy data including UUIDs, usernames, emails, and passwords.

Installation: `npm install @faker-js/faker`

Generating Fake Users Code (VS Code Style):

```
faker-generate.js

const { faker } = require('@faker-js/faker');

// Function to generate a single random user tuple
const getRandomUser = () => {
  return [
    faker.string.uuid(),           // Unique ID
    faker.internet.username(),     // Random Username
    faker.internet.email(),        // Valid-looking Email
    faker.internet.password()      // Encrypted-style password
  ];
};

// Generate 100 fake users for bulk insertion
let usersData = [];
for (let i = 0; i < 100; i++) {
  usersData.push(getRandomUser());
}
```



Bulk Insert Secret: `usersData` creates an array of arrays (tuples). The MySQL driver can insert all 100+ records in a single `INSERT INTO user VALUES ?` query in under a second!

TOPIC 03: MYSQL2 PACKAGE & CONNECTION SETUP

mysql2 Package Installation & Setup:

The fastest and most modern driver for connecting MySQL with Node.js is mysql2:

- Terminal Command: `npm install mysql2`

```
database-connection.js

const mysql = require('mysql2');

// 1. Create Connection object with credentials
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  database: 'test_db',
  password: 'mysql@123'
});

// 2. Connect to Database
connection.connect((err) => {
  if (err) throw err;
  console.log("Connected to MySQL successfully!");
});

// 3. Ending Connection (Graceful closing)
connection.end();
```

Connection Lifecycle: `end()` vs `destroy()`

- `connection.end()`: Graceful shutdown—waits for pending queries to complete before terminating the connection.
- `connection.destroy()`: Immediate termination—kills the connection socket instantly.



Production Tip (`createPool`): Rather than maintaining a single connection, production applications use `mysql.createPool()` to handle thousands of concurrent queries efficiently!

TOPIC 04: RUNNING SQL QUERIES (*connection.query*)

connection.query() Method Syntax & Execution:

To execute any SQL query against the database from Node.js, use `connection.query()`:

```
run-queries.js

const q = "SHOW TABLES";

connection.query(q, (err, result, fields) => {
  if (err) {
    console.log("SQL Query Error:", err);
    return;
  }
  console.log("Query Result:", result);
});
```

Result Format (RowDataPacket Array):

- **result:** An array of JavaScript Objects (RowDataPacket [{ id: '1', username: 'shivam' }, ...]). It can be mapped, filtered, and iterated over just like standard JS arrays.
- **fields:** Column metadata containing field names, data types, and length constraints.

Async / Await Modern Alternative (mysql2/promise):

```
promise-query.js

const mysql = require('mysql2/promise');

async function fetchUsers() {
  const connection = await mysql.createConnection({ /* config */ });
  const [rows, fields] = await connection.query("SELECT * FROM user");
  console.log(rows);
}
```

TOPIC 05: PLACEHOLDERS (?) & SQL INJECTION DEFENSE

SQL Injection Risk:



Concatenating user inputs directly into query strings ("SELECT * FROM user WHERE name = '" + name + "'"") allows attackers to inject ' OR '1'='1, which dumps or destroys the entire database!

The Solution: Placeholders (?) & Prepared Statements

Using the ? placeholder instructs the MySQL2 driver to treat user input strictly as **data**, preventing it from ever being parsed as executable SQL code!

Single vs Bulk Insert Code:

```
insert-placeholders.js

-- 1. SINGLE INSERT (Using individual ? placeholders)
const q1 = "INSERT INTO user (id, username, email, password) VALUES (?, ?, ?, ?)";
const userData = ['101', 'shivam', 'shivam@test.com', 'pass123'];

connection.query(q1, userData, (err, res) => {
  if (err) throw err;
  console.log("1 User inserted!");
});

-- 2. BULK INSERT (Using single ? for Array of Arrays)
const q2 = "INSERT INTO user (id, username, email, password) VALUES ?";
connection.query(q2, [userData], (err, res) => {
  if (err) throw err;
  console.log(res.affectedRows + " users inserted in 1 query!");
});
```

TOPIC 06: RESTFUL ROUTING WITH MYSQL (5 CORE ROUTES)

5 Core RESTful Routes for User Resource:

HTTP Method	Route Path	Purpose & Action	SQL Query Executed
GET	/	Home: Show total count of users in database	SELECT COUNT(*) FROM user
GET	/user	Index: List all users (id, username, email)	SELECT id, username, email FROM user
POST	/user	Create: Insert a new user into database	INSERT INTO user VALUES (?, ?, ?, ?)
PATCH	/user/:id	Update: Modify a specific user's username	UPDATE user SET username = ? WHERE id = ?
DELETE	/user/:id	Destroy: Remove a specific user from database	DELETE FROM user WHERE id = ?

Form Routes (Supporting HTML Views):

- GET /user/new: Renders the HTML registration form for adding a new user.
- GET /user/:id/edit: Renders the editing form for a specific user (pre-filling existing data).

TOPIC 07: EXECUTING SQL FILES VIA CLI (SCHEMA.SQL)

schema.sql File Workflow:

In production applications, rather than hardcoding SQL statements in JS code, database tables are structured inside a clean `schema.sql` file:

```
CREATE TABLE IF NOT EXISTS user (  
  id VARCHAR(50) PRIMARY KEY,  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password VARCHAR(50) NOT NULL  
);
```

`schema.sql`

Steps to Execute schema.sql via CLI:

- **Step 1:** Log in to MySQL via Terminal / Command Prompt:
`mysql -u root -p` (Enter password)
- **Step 2:** Select the database:
`USE test_db;`
- **Step 3:** Execute the file:
`source schema.sql; or source /path/to/schema.sql;`



One-Line Terminal Shortcut: Import the entire schema directly from the terminal without entering the interactive

shell:

```
mysql -u root -p database_name < schema.sql
```

TOPIC 08: COMPLETE EXPRESS + MYSQL REST SERVER

Full Production-Ready index.js (VS Code Style):

```
index.js - Express + MySQL RESTful Architecture

const express = require('express');
const app = express();
const mysql = require('mysql2');
const methodOverride = require('method-override');

app.use(express.urlencoded({ extended: true }));
app.use(methodOverride('_method'));
app.set('view engine', 'ejs');

const connection = mysql.createConnection({
  host: 'localhost', user: 'root', database: 'test_db', password: 'mysql@123'
});

// 1. GET / → Show count of users
app.get('/', (req, res) => {
  connection.query('SELECT COUNT(*) as count FROM user', (err, result) => {
    if (err) throw err;
    res.render('home.ejs', { count: result[0].count });
  });
});

// 2. GET /user → Show all users
app.get('/user', (req, res) => {
  connection.query('SELECT id, username, email FROM user', (err, users) => {
    if (err) throw err;
    res.render('users.ejs', { users });
  });
});

// 3. PATCH /user/:id → Edit username
app.patch('/user/:id', (req, res) => {
  const { id } = req.params;
  const { username } = req.body;
  connection.query('UPDATE user SET username = ? WHERE id = ?', [username, id], (err) => {
    if (err) throw err;
    res.redirect('/user');
  });
});

// 4. DELETE /user/:id → Delete user
app.delete('/user/:id', (req, res) => {
  connection.query('DELETE FROM user WHERE id = ?', [req.params.id], (err) => {
    if (err) throw err;
    res.redirect('/user');
  });
});

app.listen(8080, () => console.log("Server listening on port 8080"));
```